



AI-Powered Task Management System

A full-stack task management system enhanced with AI capabilities for intelligent task analysis, built using **Spring Boot, React, and FastAPI**.



Overview

This project goes beyond a typical CRUD application by integrating an AI microservice to automatically:

- Suggest task priority
- Estimate effort
- Generate task summaries

It demonstrates **real-world system design**, including authentication, microservice communication, and full-stack integration.



Architecture

React (Frontend) ↓ Spring Boot (Backend - JWT Secured APIs) ↓ FastAPI (AI Microservice)



Tech Stack

Backend

- Java
- Spring Boot
- Spring Security (JWT Authentication)
- JPA / Hibernate

Frontend

- React
- JavaScript
- Axios

AI Service

- Python
 - FastAPI
-

Key Features

Authentication & Security

- JWT-based authentication
 - Protected REST APIs
 - Stateless session handling
 - Global error handling (403 interceptor)
-

AI-Powered Enhancements

- Automatic **priority prediction**
 - AI-generated **task summary**
 - Estimated **task effort**
 - Backend-to-microservice communication (Java ↔ Python)
-

Task Management

- Create, delete, and view tasks
 - Status lifecycle management (TODO, IN_PROGRESS, DONE)
 - Filtering by:
 - Status
 - Priority
-

Frontend Capabilities

- Login/logout flow
 - Token-based API communication
 - Dynamic filtering
 - Error handling and recovery
 - Clean UI rendering
-

System Flow

User logs in → JWT generated

↓

Token stored in browser

↓

Frontend sends requests with token

↓

Spring Boot validates JWT

↓

Task created → AI service called

↓

AI enriches task (priority, summary, effort)

↓

Data stored and returned to UI

Sample API Flow

◆ Login

POST /auth/login

◆ Create Task

POST /tasks

Authorization: Bearer <token>

◆ Get Tasks

GET /tasks

Authorization: Bearer <token>

Key Learnings

- Implemented **JWT authentication with Spring Security**
 - Built **stateless authentication flow**
 - Integrated **Java backend with Python microservice**
 - Handled **CORS and security filter chain issues**
 - Managed **frontend-backend data contracts**
 - Debugged real-world issues like:
 - Token invalidation
 - API response mapping
 - Async rendering bugs
-

Challenges Solved

- JWT key mismatch causing 403 errors
 - Nested API response handling (Page<T> mapping)
 - CORS issues with Spring Security
 - Frontend state inconsistency during async calls
 - Microservice data mapping mismatches
-

Screenshots

Add screenshots here

- Login Page
 - Task Dashboard
 - AI-enriched task display
-

Setup Instructions

Backend (Spring Boot)

```
mvn spring-boot:run
```

Frontend (React)

```
npm install  
npm start
```

AI Service (FastAPI)

```
uvicorn main:app --reload
```

Future Improvements

- Role-based authorization
 - Task update functionality
 - Better UI/UX (toasts, loaders)
 - Deployment (cloud hosting)
 - Token refresh mechanism
-

Resume Highlight

Developed a full-stack AI-powered task management system integrating Spring Boot, React, and FastAPI with JWT-based authentication and real-time task enrichment.